Estructuras de Datos
Clase 2 – Pilas y colas

Dr. Sergio A. Gómez
http://cs.uns.edu.ar/~sag

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Temas de hoy

- Excepciones
  - Try-catch
  - Ejemplos con tipos conocidos
- Tipo de dato abstracto
- Pila de java.util.Stack
- Cola de java.util.Queue

Haciendo programa robusto: Excepciones

- Las excepciones son eventos inesperados que ocurren durante la ejecución de un programa.
- Puede ser el resultado de una condición de error o una entrada incorrecta.
- En POO las excepciones son objetos “lanzados” por un código que detecta una condición inesperada.
- La excepción es “capturada” por otro código que repara la situación
- Si la excepción no es capturada, entonces se propaga al método llamante y así sucesivamente hasta que algún método capture la excepción con un try-catch.
- Si el main no captura la excepción, el programa termina su ejecución con error.

Excepciones: try-catch

```
void método( parámetros formales )
{
    try
    {
            … otras sentencias que no producen error ...
            sentencia_que_puede_producir_excepción();
            … más sentencias que no producen error….
    }
    catch( tipo_excepcion e )
    {
            código para manejar la excepción e
    }
}
```

SI NO HAY ERROR, NO SE EJECUTA EL CODIGO DEL CATCH.

Excepciones: Ejemplo

Problema: Leer un entero “pos” y mostrar la componente “pos” del arreglo “a”, el doble y el triple.

```
import java.util.Scanner;
import java.io.*;

public class EjExcepcion01
{
    public static void main( String [] args )
    {
        try {
            int [] a = { 1, 2, 3, 4, 5 };
            Scanner s = new Scanner( System.in );
            System.out.print( "Ingrese una posicion entre 0 y 4: " );
            int pos = s.nextInt();
            System.out.println( "El valor en " + pos + " es " + a[pos] );
            System.out.println( "El doble es " + 2 * a[pos] );
            System.out.println( "El triple es " + 3 * a[pos] );
        } catch( ArrayIndexOutOfBoundsException e ) {
            System.out.println( "Valor fuera de rango" );
        }
    }
}
```

```
>java EjExcepcion01
Ingrese una posicion entre 0 y 4: 40
Valor fuera de rango
```

Excepciones: try-catch-finally

Puedo usar más de un catch para capturar varias excepciones. Puedo usar un bloque finally para ejecutar un código independientemente de que haya o no error.

```
try   {
        …….
}  catch( tipo_excepcion_1 e ) {
        código para manejar la excepción e
} catch( tipo_excepcion_2 e ) {
        código para manejar la excepción e
} finally {
      código que siempre se ejecuta
}
```

Excepciones: Ejemplo

```
try {
    int [] a = { 1, 2, 3, 4, 5 };
    Scanner s = new Scanner( System.in );
    System.out.print( "Ingrese una posicion entre 0 y 4: " );
    int pos = s.nextInt();
    System.out.println( "El valor en " + pos + " es " + a[pos] );
    System.out.println( "El doble es " + 2 * a[pos] );
    System.out.println( "El triple es " + 3 * a[pos] );
} catch( ArrayIndexOutOfBoundsException e ) {
    System.out.println( "Valor fuera de rango" );
} catch( InputMismatchException e ) {
    System.out.println( "No ingreso un entero" );
} finally {
    System.out.println( "Adios!" );
}
```

```
>java EjExcepcion02
Ingrese una posicion entre 0 y 4: mama
No ingreso un entero
Adios!

>java EjExcepcion02
Ingrese una posicion entre 0 y 4: 2
El valor en 2 es 3
El doble es 6
El triple es 9
Adios!
```

Notas: Excepciones checked y unchecked

- Las excepciones como ArrayIndexOutOfBoundsException se llaman *unchecked* porque el programa compilará aunque el cliente no tengan un try-catch asociado. Esto ocurre con las RuntimeException's.
- Las excepciones subclase de Exception se llaman *checked* porque el cliente no compilará si el código cliente no está protegido con el try-catch correspondiente.

Notas

- Las excepciones se estructuran en una jerarquía de herencia, al utilizar cláusulas catch múltiples, en el caso de una excepción éstas se comprueban en forma secuencial hasta que la el tipo de la excepción lanzada conforma al de la excepción declarada en el catch y recién en ese momento se ejecuta este último bloque catch.
- Entonces estructuras los catch de excepción más específica a más general.

Ejemplo: El código del bloque try puede lanzar una IOExcepcion (que es un tipo particular de Exception) o alguna otra excepción de otro tipo que sea subclase de Exception.

El primer catch captura IOException y el segundo todos los otros tipos de excepciones ya sean de tipo Exception o alguna de sus subclases.

```
try {
  // Código que puede lanzar una excepción
  ....
} catch( IOException e ) {
  // Código para procesar la excepción de entrada / salida
  ...
} catch( Exception e ) {
 // Código para procesar un excepción general
 ...
}
```

Ejemplo de código incorrecto:

Como IOException es subclase de Exception, el manejador de IOException es inalcanzable (esto lo marcan los compiladores modernos).

```
try {
    ....
} catch( Exception e ) {
    ....
} catch( IOException e ) {  // Código inalcanzable ya que esta
// excepción es capturada siempre por el bloque catch
anterior.
...
}
```

Algunas excepciones unchecked de Runtime definidas en java.lang

Runtime exception	Significado
ArithmeticException	Error de aritmética, e.g. división por cero
ArrayIndexOutOfBoundsException	Indice de arreglo fuera de límites
ClassCastException	Cast inválido
IndexOutOfBoundsException	Algún índice de array fuera de límites
NegativeArraySizeException	Array creado con tamaño negativo
NullPointerException	Uso incorrecto de una referencia nula
NumberFormatException	Conversión inválida de string en número
StringIndexOutOfBounds	Se quiso acceder a una posición inexistente de un string

Métodos útiles definidos por Throwable (superclase de Exception)

Método	Significado
String getMessage()	Función que retorna una descripción de la excepción
void printStackTrace()	Procedimiento que imprime a la consola la pila de registros de activación con nombre de método y número de línea en cada método donde se propagó la excepción hasta que fue capturada.
String toString()	Retorna una descripción de la excepción

<u>Ejemplo de uso:</u>

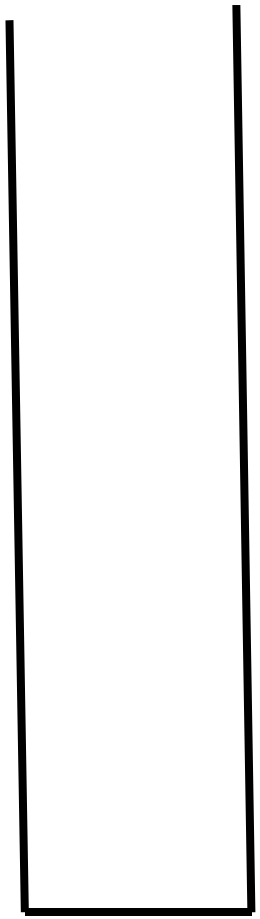
```
try {
    … código que puede lanzar una excepción de tipo MiExcepción ….
} catch ( MiExcepción e ) {
    System.out.println( e.getMessage() );  // Imprime mensaje asociado a e
    e.printStackTrace();  // Imprime pila de registros de activación indicando en qué
línea de Código falló cada método.
}
```

Tipo de dato abstracto

- Un *tipo de dato abstracto* (TDA) (en inglés, ADT por Abstract Data Type) es un tipo definido solamente en términos de sus operaciones y de las restricciones que valen entre las operaciones.
- Las restricciones las daremos en términos de comentarios.
- Cada TDA se representa en esta materia con una (o varias) interfaces.
- Una o más implementaciones del TDA se brindan en términos de clases concretas.
- Hoy daremos los TDAs Pila (Stack) y Cola (Queue).

Tipo de dato abstracto Pila

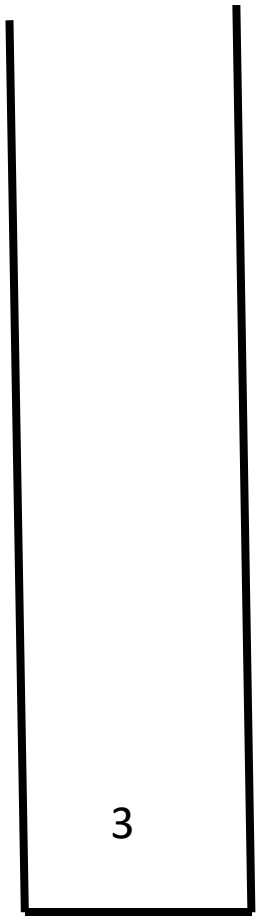
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Esta es una pila vacía

Tipo de dato abstracto Pila

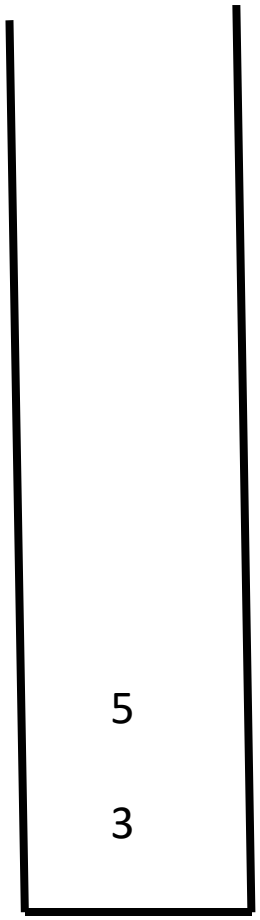
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3

Tipo de dato abstracto Pila

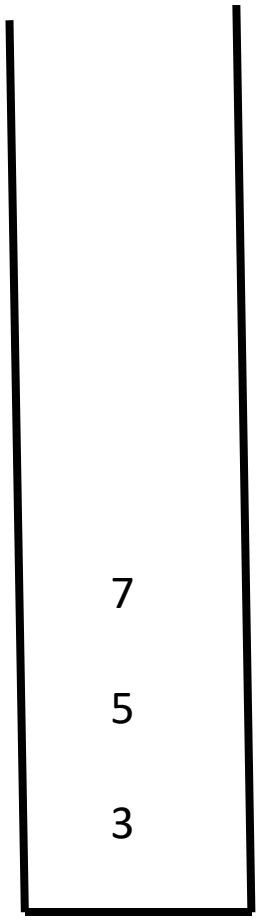
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5

Tipo de dato abstracto Pila

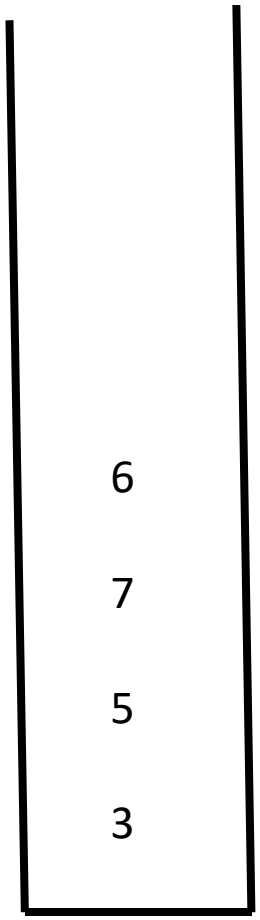
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7

Tipo de dato abstracto Pila

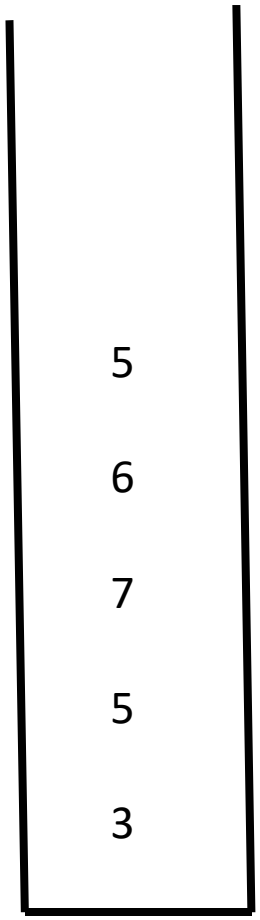
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6

Tipo de dato abstracto Pila

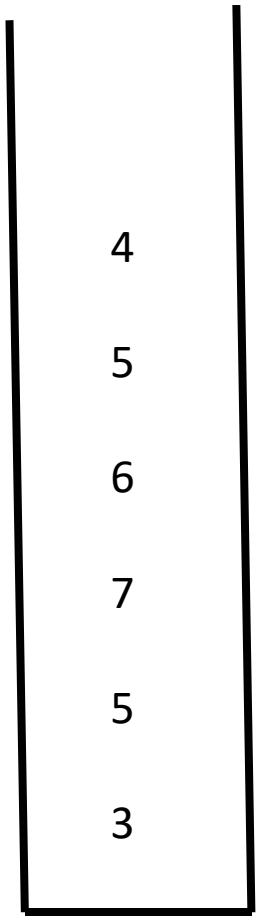
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6
Apilar 5

Tipo de dato abstracto Pila

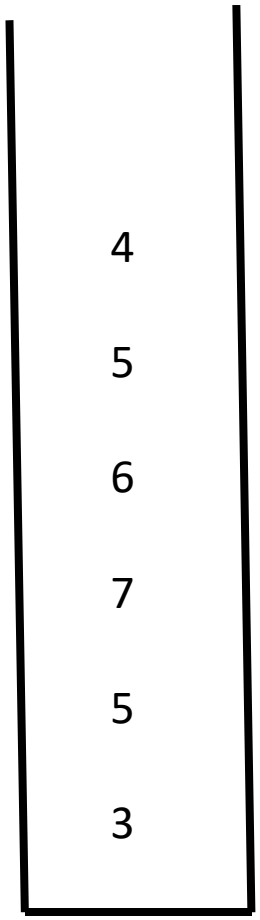
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6
Apilar 5
Apilar 4

Tipo de dato abstracto Pila

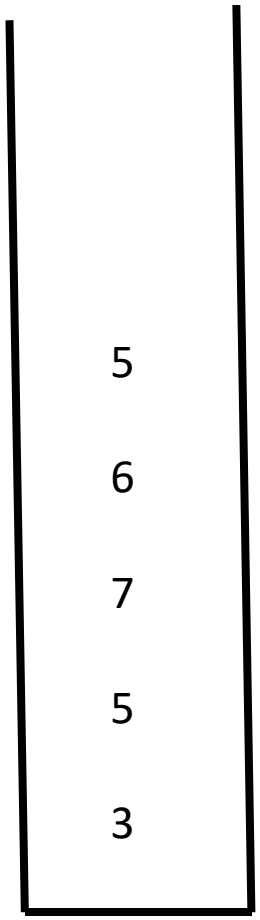
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6
Apilar 5
Apilar 4
El tope de la pila es 4

Tipo de dato abstracto Pila

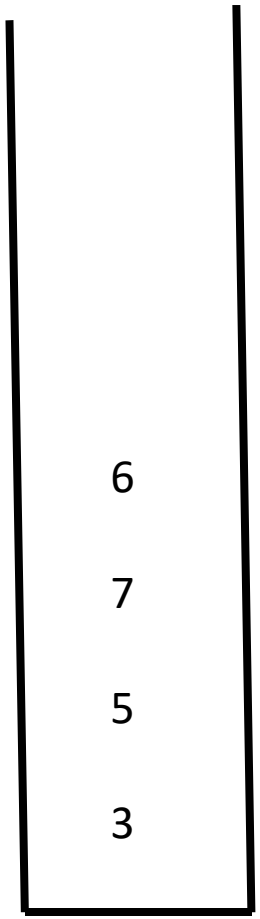
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6
Apilar 5
Apilar 4
El tope de la pila es 4
Desapilar retorna 4 y lo elimina de la pila
El tope de la pila es 5

Tipo de dato abstracto Pila

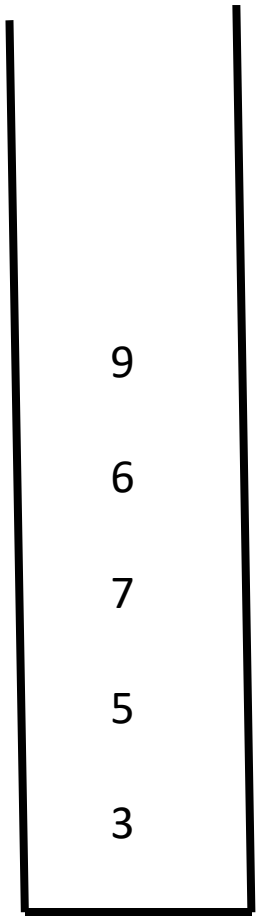
<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6
Apilar 5
Apilar 4
El tope de la pila es 4
Desapilar retorna 4 y lo elimina de la pila
El tope de la pila es 5
Desapilar retorna 5 y lo elimina de la pila

Tipo de dato abstracto Pila

<u>Pila:</u> Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
Apilar 5
Apilar 7
Apilar 6
Apilar 5
Apilar 4
El tope de la pila es 4
Desapilar retorna 4 y lo elimina de la pila
El tope de la pila es 5
Desapilar retorna 5 y lo elimina de la pila
Apilar 9
El tope de la pila es 9

TDA Pila (Stack)

Operaciones:

- apilar(e): Comando que inserta el elemento e en el tope de la pila
- desapilar(): Comando que elimina el elemento del tope de la pila y lo entrega como resultado. Si se aplica a una pila vacía, hay un error.
- vacía(): Consulta que retorna verdadero si la pila no contiene elementos y falso en caso contrario.
- tope(): Consulta que retorna el elemento del tope de la pila. Si se aplica a una pila vacía, hay un error.
- tamaño(): Consulta que retorna un entero que indica cuántos elementos hay en la pila.

Implementación de Pila en java.util

- Hay una clase que se llama java.util.Stack
- Tiene un parámetro de tipo para especificar el tipo de los elementos de la pila: Stack<E>
- Ejemplos de instancias de pilas:
  - Pila de strings: Stack<String>
  - Pila de enteros: Stack<Integer>
  - Pila de flotantes: Stack<Float>
  - Pila de personas: Stack<Persona>
  - Pila de pilas de enteros: Stack<Stack<Integer>>

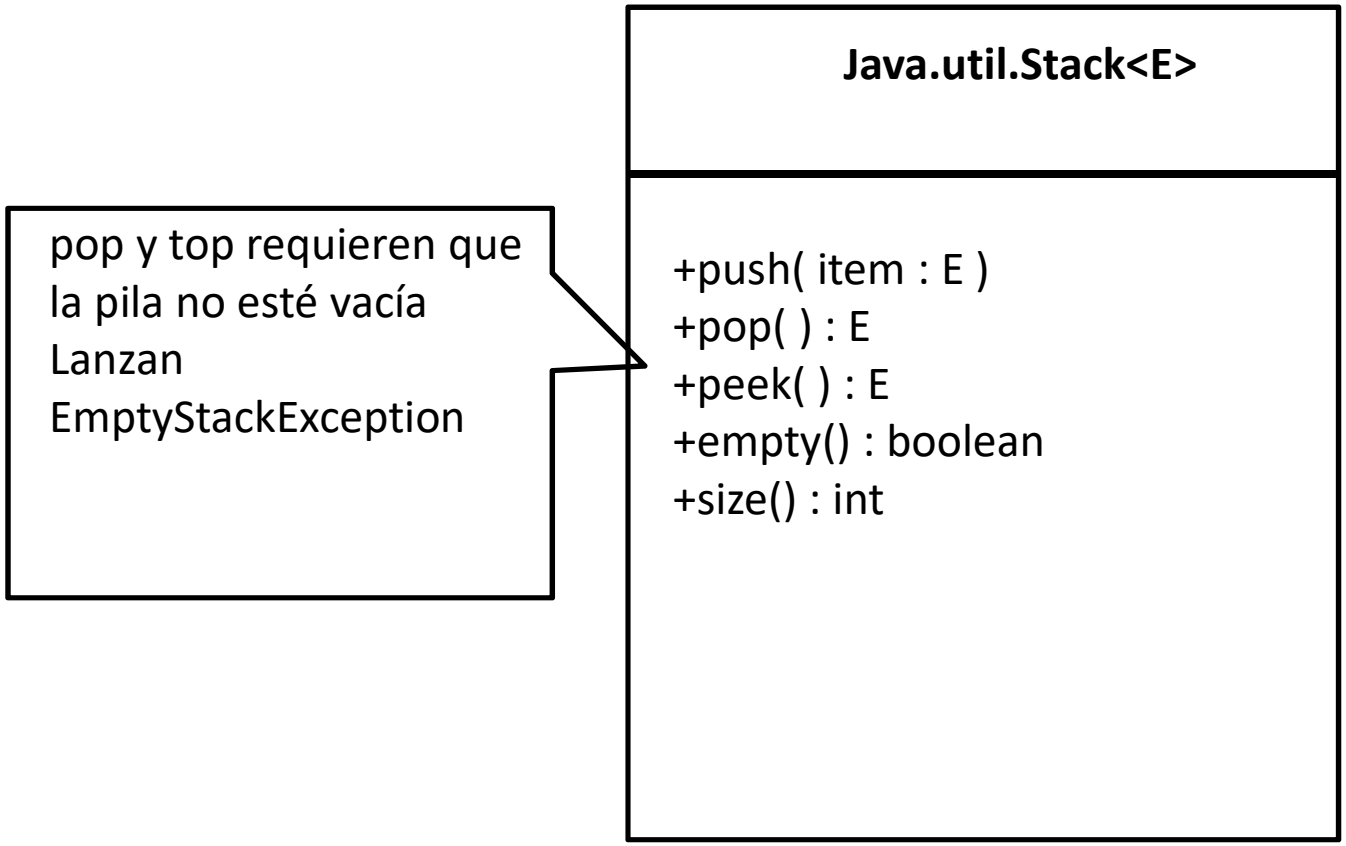
Implementación del TDA Pila: java.util.Stack

Operaciones:

- push(e): Comando que inserta el elemento e en el tope de la pila
- pop(): Comando que elimina el elemento del tope de la pila y lo entrega como resultado. Si se aplica a una pila vacía, lanza EmptyStackException.
- empty(): Consulta que retorna verdadero si la pila no contiene elementos y falso en caso contrario.
- peek(): Consulta que retorna el elemento del tope de la pila. Si se aplica a una pila vacía, lanza EmptyStackException.
- size(): Consulta que retorna un entero que indica cuántos elementos hay en la pila.

Especificación de Stack en UML

En UML las operaciones se dan con una sintaxis Pascal-like y las excepciones se dan como comentarios:



Javadoc de java.util.Stack

https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html

Constructor Summary

Constructors

Constructor and Description
`Stack()` Creates an empty Stack.

Method Summary

All Methods | **Instance Methods** | **Concrete Methods**

Modifier and Type	Method and Description
`boolean`	`empty()` Tests if this stack is empty.
`E`	`peek()` Looks at the object at the top of this stack without removing it from the stack.
`E`	`pop()` Removes the object at the top of this stack and returns that object as the value of this function.
`E`	`push(E item)` Pushes an item onto the top of this stack.
`int`	`search(Object o)` Returns the 1-based position where an object is on this stack.

EJEMPLO DE USO DE LA PILA DE JAVA:

```
// Importar clase pila y excepción de pila vacía:
import java.util.EmptyStackException;
import java.util.Stack;

public void testearPila() {
    try {
        // Crear una pila de strings vacía
        Stack<String> pilaStr = new Stack<String>();
        // Apilar Sergio:  pilaStr = ["Sergio"]
        pilaStr.push("Sergio");
        // Apilar Martin: pilaStr = ["Sergio", "Martin"]
        pilaStr.push("Martin");
         // Apilar Matias: pilaStr = ["Sergio", "Martin", "Matias"]
        pilaStr.push("Matias");
        // Imprimir la pila: Imprime pilaStr = ["Sergio", "Martin", "Matias"]
        System.out.println("pilaStr: " + pilaStr);

        // Mirando el tope de la pila: tope = "Matias"
        String tope = pilaStr.peek();
        System.out.println("tope: " + tope);
```

EJEMPLO DE USO DE LA PILA DE JAVA (Continuación)

```
boolean estaVacia = pilaStr.empty();  // estaVacia = false
System.out.println("Esta vacia: " + estaVacia);

// Desapilar un elemento:
String elemento = pilaStr.pop();
System.out.println("elemento: " + elemento);
// pilaStr = ["Sergio", "Martin"]
System.out.println("pila luego de desapilar elemento: " + pilaStr);
```

EJEMPLO DE USO DE LA PILA DE JAVA (Continuación bis)

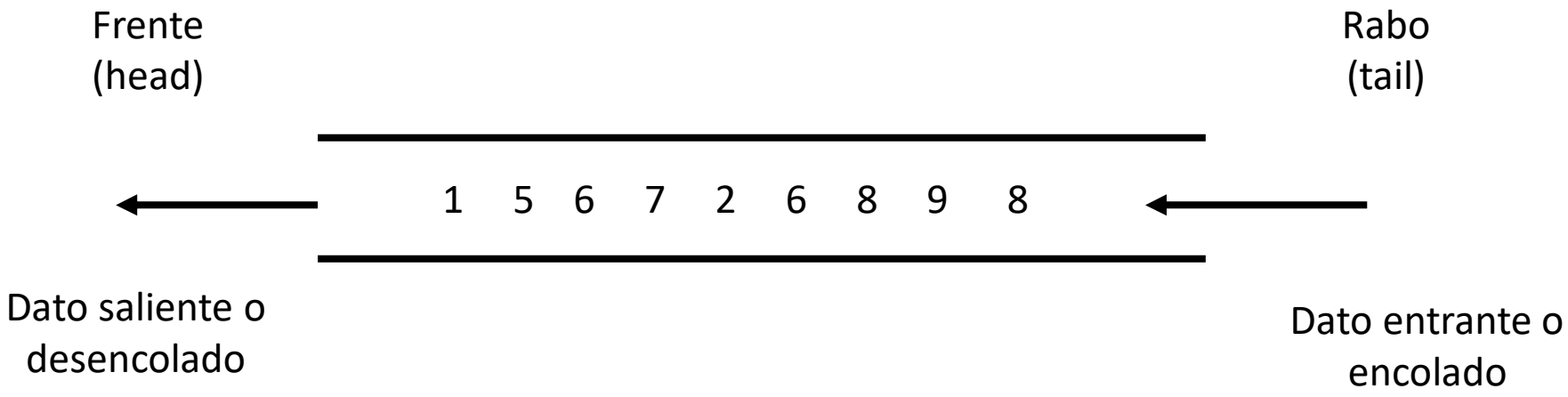
```
        // Vaciar pila: Salen Martin y luego Sergio (en ese orden).
        System.out.println("Vaciar la pila:");
        while ( pilaStr.empty() == false ) {
                elemento = pilaStr.pop();
                System.out.println("elemento: " + elemento);
        }

        estaVacia = pilaStr.empty();  // estaVacia = true
        System.out.println("Esta vacia: " + estaVacia);

        System.out.println("Producir un error a proposito");
        pilaStr.pop();

        System.out.println("Este codigo es inalcanzable");
} catch( EmptyStackException e ) {
        System.out.println("Hubo un intento de desapilar de una pila vacia.");
        System.out.println("Info error: " + e.getMessage());
}
}  // Fin método
```

TDA Cola



Una *cola* (queue) es una colección con modelo de secuencia cuyos datos ingresan por un extremo llamado *rabo* (tail) y salen por otro extremo llamado *frente* (head).

Esta política de actualización se llama FIFO (por *first-in first-out*) o también FCFS (*first-come first-served*).

TDA Cola

<u>Operaciones:</u>

- encolar(e): Inserta el elemento *e* en el rabo de la cola
- desencolar(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía, se produce un error.
- frente(): Retorna el elemento del frente de la cola. Si la cola está vacía, hay un error
- vacía(): Retorna verdadero si la cola no tiene elementos y falso en caso contrario
- tamaño(): Retorna la cantidad de elementos de la cola.

Implementación de Cola en java.util

- Hay una interfaz java.util.Queue
- Las operaciones de esta interfaz son implementadas por java.util.LinkedList
- Ofrece dos filosofías para señalar errores:

Operación	*Lanza excepción*	*Devuelve valor especial*
Encolar	add(e)	offer(e)
Desencolar	remove()	poll()
Frente	element()	peek()

(Captura de Javadoc en siguiente diapositiva)

Interfaz java.util.Queue

https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html

Interface Queue<E>

Type Parameters:

E - the type of elements held in this collection

All Superinterfaces:

Collection<E>, Iterable<E>

Method Summary

All Methods | Instance Methods | Abstract Methods

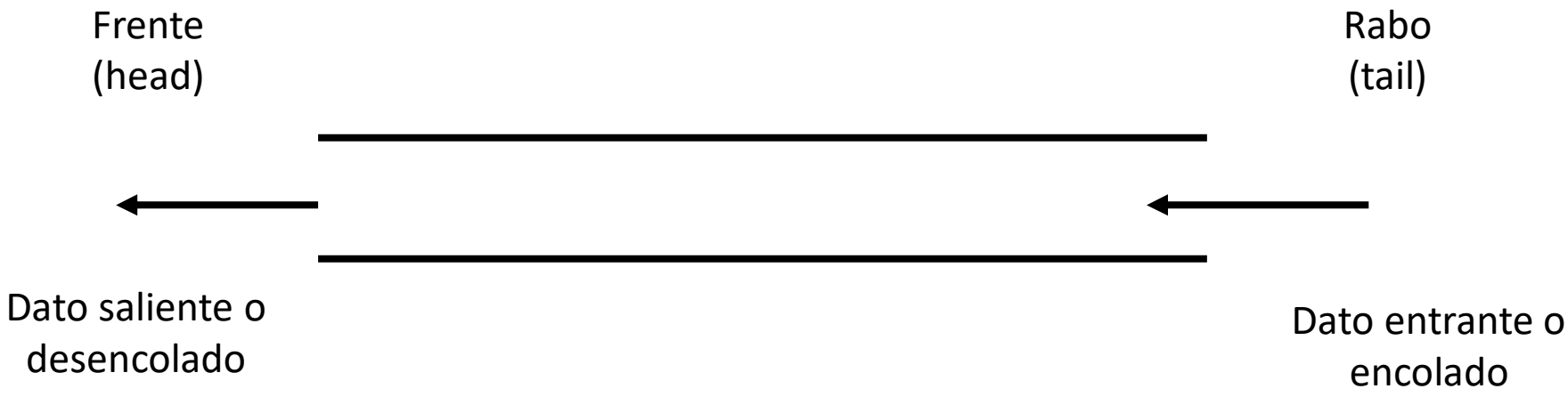
Modifier and Type	Method and Description
boolean	**add(E e)** Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.
E	**element()** Retrieves, but does not remove, the head of this queue.
boolean	**offer(E e)** Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions.
E	**peek()** Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
E	**poll()** Retrieves and removes the head of this queue, or returns `null` if this queue is empty.
E	**remove()** Retrieves and removes the head of this queue.

TDA Cola (con excepciones)

<u>Operaciones:</u>

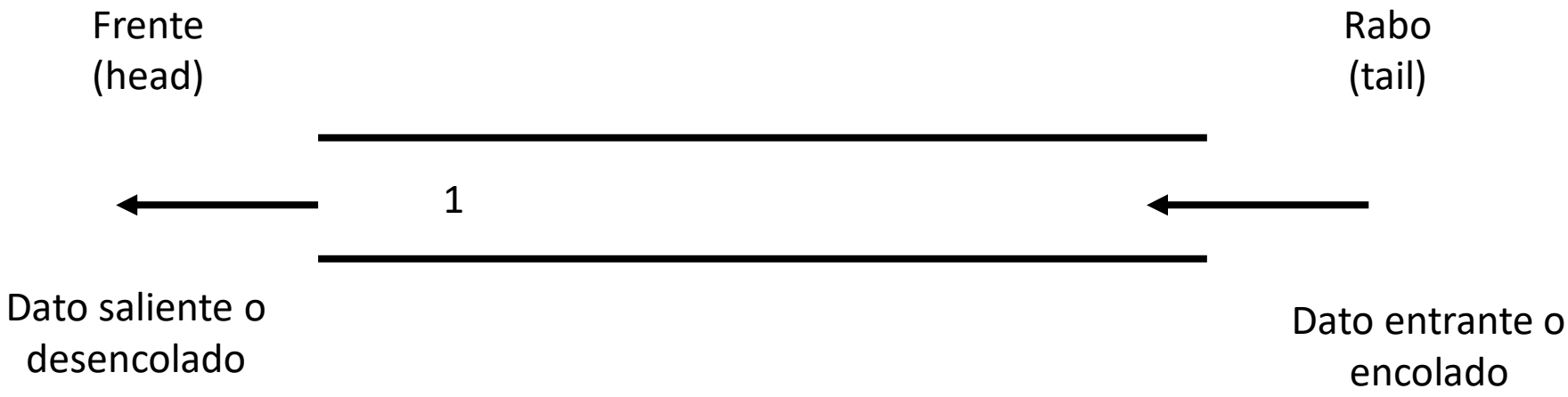
- add(e): Inserta el elemento *e* en el rabo de la cola
- remove(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía se lanza una excepción NoSuchElementException.
- element(): Retorna el elemento del frente de la cola. Si la cola está vacía se lanza una NoSuchElementException
- isEmpty(): Retorna verdadero si la cola no tiene elementos y falso en caso contrario
- size(): Retorna la cantidad de elementos de la cola.

TDA Cola



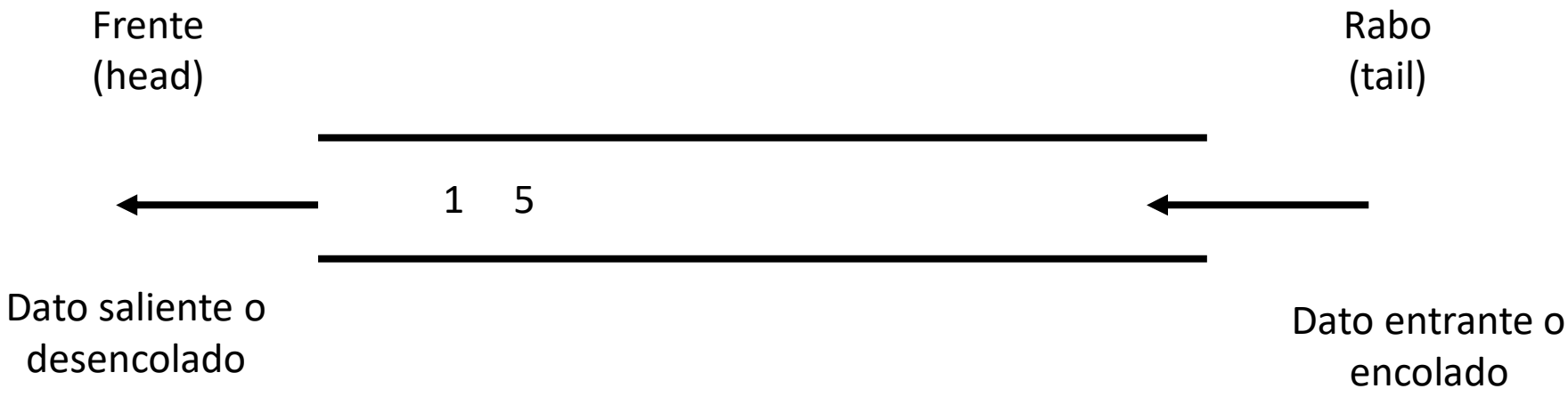

```
Queue<Integer> q = new LinkedList<Integer>();
```

TDA Cola



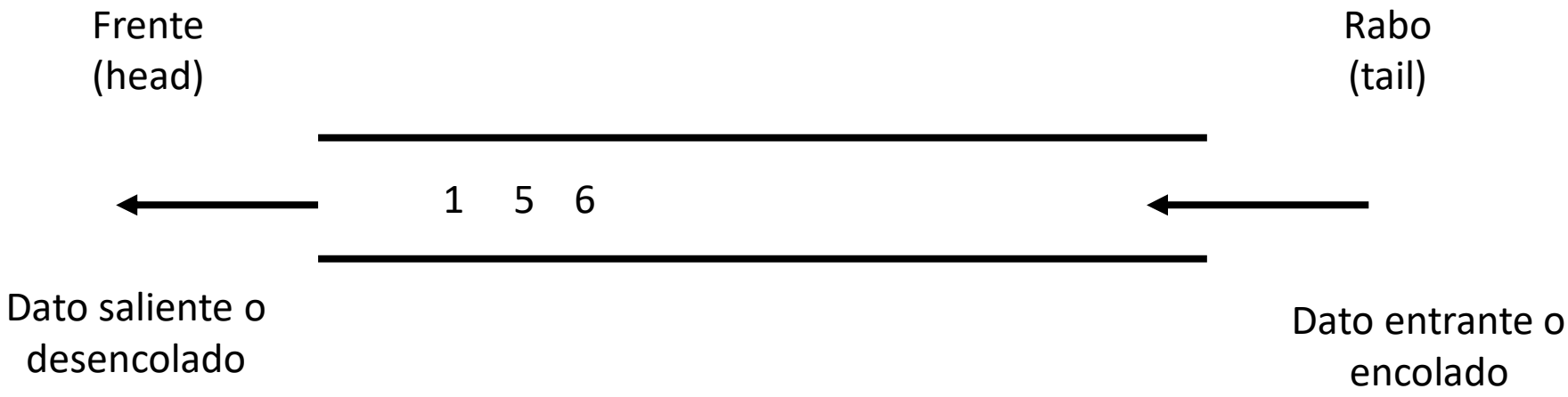

```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
```

TDA Cola



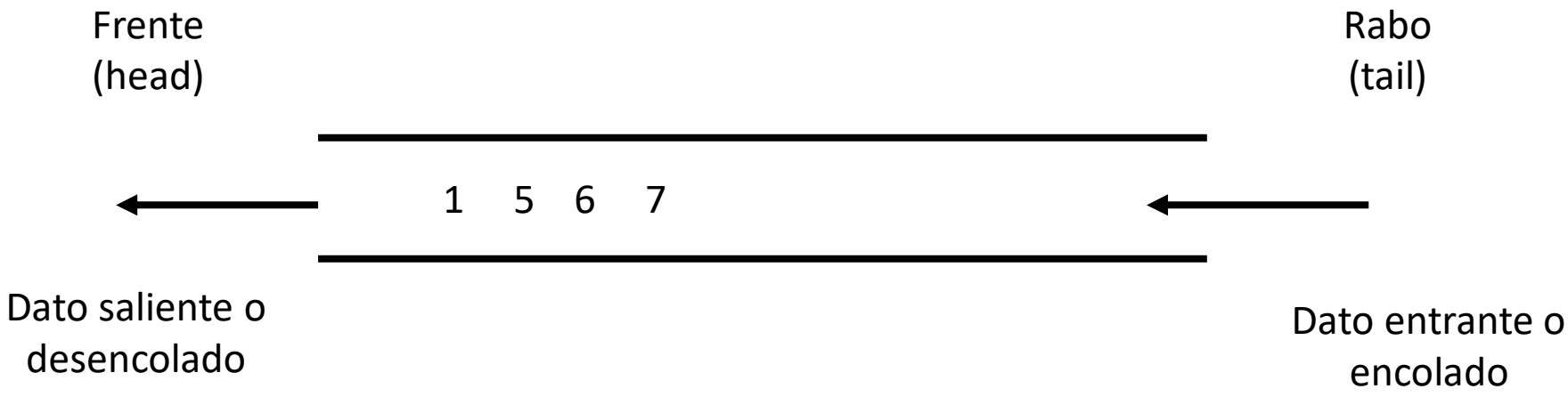

```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
q.add(5);
```

TDA Cola



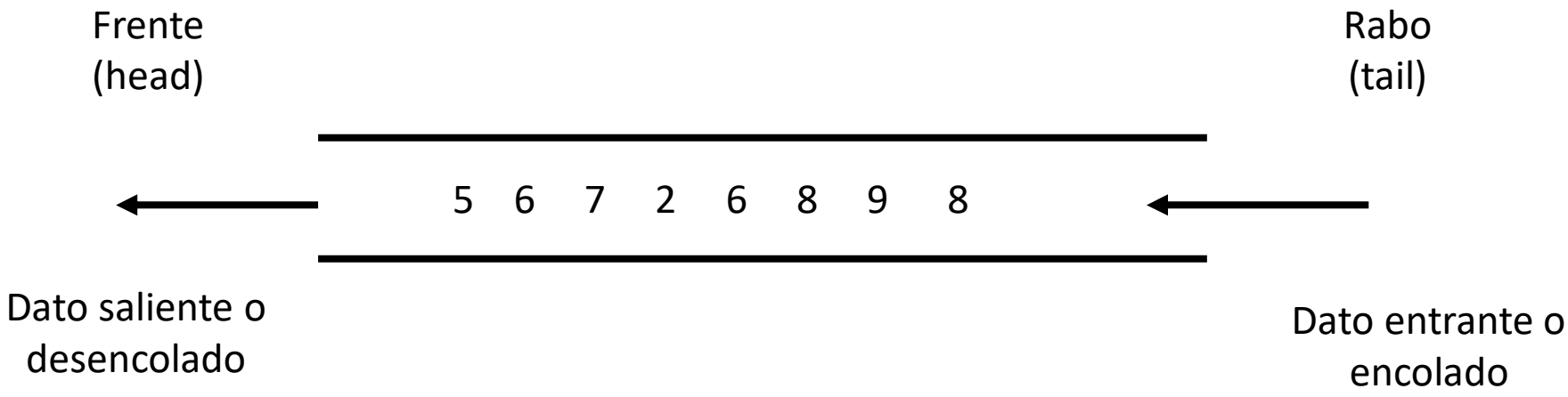

```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
q.add(5);
q.add(6);
```

TDA Cola



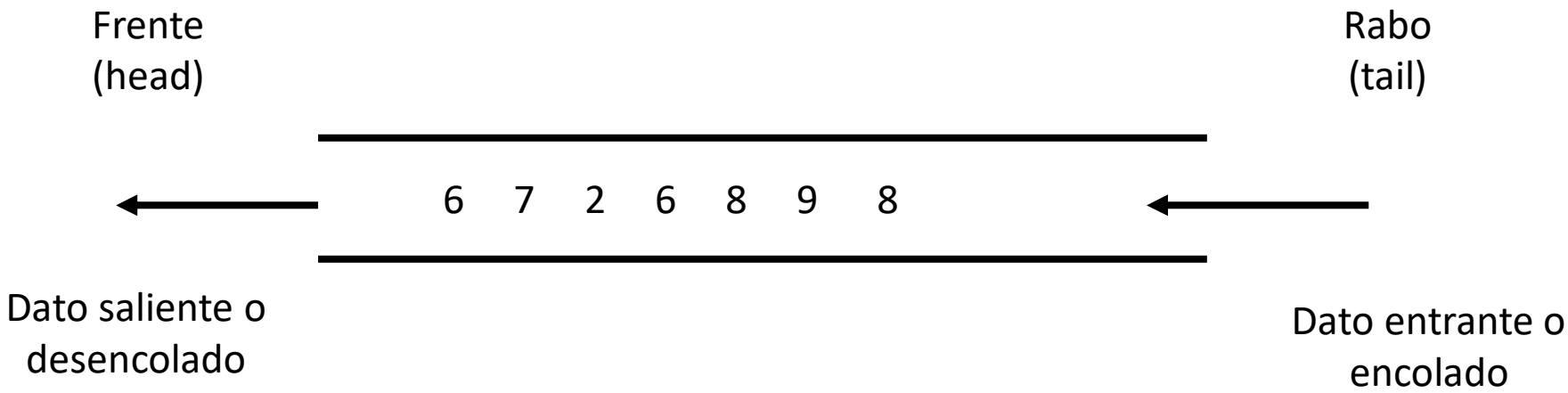

```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
q.add(5);
q.add(6);
q.add(7);
```

TDA Cola



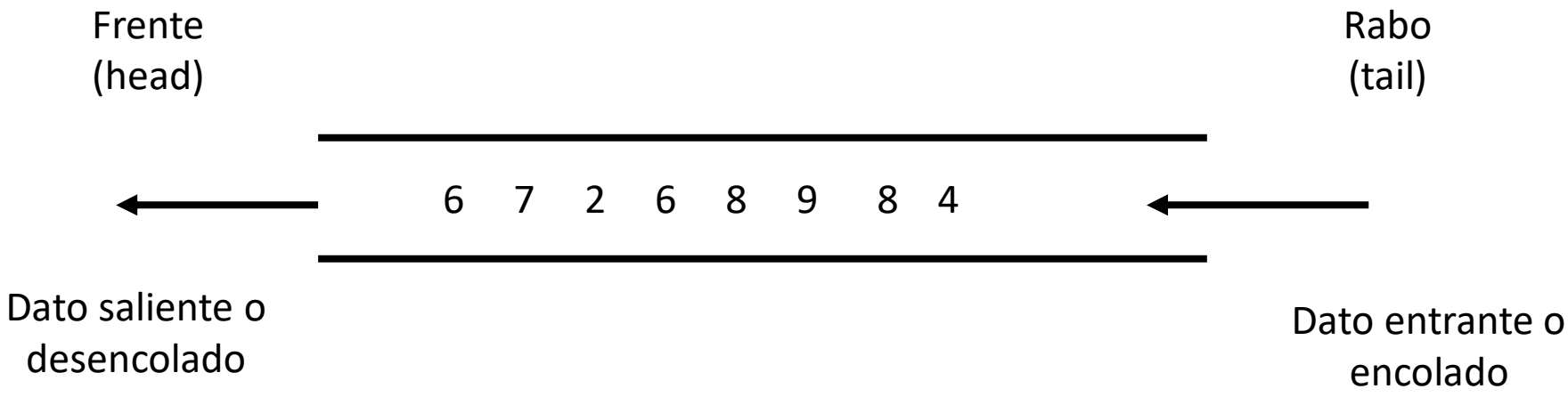

```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
q.add(5);
q.add(6);
q.add(7);
….
q.add(8);
int elemento = q.remove();   // elemento vale 1 y se borra de la cola
```

TDA Cola




```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
q.add(5);
q.add(6);
q.add(7);
….
q.add(8);
int elemento = q.remove();   // elemento vale 1 y se borra de la cola
q.remove();                  // borro el 5 de la cola y no lo almaceno
```

TDA Cola




```
Queue<Integer> q = new LinkedList<Integer>();
q.add(1);
q.add(5);
q.add(6);
q.add(7);
….
q.add(8);
int elemento = q.remove();   // elemento vale 1 y se borra de la cola
q.remove();                  // borro el 2 de la cola y no lo almaceno
q.add(4);                    // Puedo encolar cuando quiera
```

EJEMPLO DE USO DE COLA DE JAVA

```
import java.util.Queue;
import java.util.LinkedList;

public void testearCola() {
try {
        // Creo una cola de enteros y la instancio con una lista enlazada.
        Queue<Integer> cola = new LinkedList<Integer>();
        // Encolo 3, 5, 8, 2
        cola.add(3);
        cola.add(5);
        cola.add(8);
        cola.add(2);
        // Imprimo cantidad de elementos: 4
        System.out.println("Cantidad de elementos de la cola: " + cola.size() );
        System.out.println("Cola: " + cola);
        System.out.println("Cola vacia?: " + cola.isEmpty()); // falso
        System.out.println("Frente de cola: " + cola.peek());  // 3
        System.out.println("Vamos a desencolar:");
        int primero = cola.remove(); // Desencolo el 3
        System.out.println("Primero: " + primero);  // primero = 3
```

EJEMPLO DE USO DE COLA DE JAVA (continuación)

```
System.out.println("Cola: " + cola); // cola = [5, 8, 2]
int segundo = cola.remove(); // Desencolo el 5
System.out.println("segundo: " + segundo); // segundo = 5
System.out.println("Cola: " + cola); // cola = [8, 2]
System.out.println("Vamos a vaciar la cola:");
while ( cola.isEmpty() == false ) {
        cola.remove();
}
System.out.println("Cola vacia?: " + cola.isEmpty()); // true
System.out.println("Voy a ver si puedo ver el primer elemento de una cola vacia");
int elementoInexistente = cola.element(); // Esta sentencia produce excepción NoSuchElementException
} catch ( java.util.NoSuchElementException e) {
        e.printStackTrace();
}
}
```

TDA Cola (con valores especiales)

<u>Operaciones:</u>

- offer(e): Inserta el elemento *e* en el rabo de la cola. Retorna true si pudo insertar y false si no pudo insertar porque se llenó la lista.
- poll(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía, devuelve null.
- peek(): Retorna el elemento del frente de la cola. Si la cola está vacía, devuelve null.
- isEmpty(): Retorna verdadero si la cola no tiene elementos y falso en caso contrario
- size(): Retorna la cantidad de elementos de la cola.

Ejemplo de la cola Java usando valores especiales (con offer, poll y peek). Inserta 3, 6, 8, 7 y luego eliminarlos e imprimirlos.
Requiere un estilo de programación más cercano a la programación en C (lenguaje del que deriva Java).

```
import java.util.Queue;
import java.util.LinkedList;

public class ColaSimple {
   public static void main(String[] args) {
      Queue<Integer> cola = new LinkedList<>();

      // Insertar elementos (offer)
      cola.offer(3);
      cola.offer(6);
      cola.offer(8);
      cola.offer(7);

      // Procesar la cola usando valor especial (null)
      Integer elemento;

      while ((elemento = cola.poll()) != null) {
         System.out.println("Eliminado: " + elemento);
      }
   }
}
```

PRESTAR ATENCIÓN A LA FORMA DE LA CONDICION

Hace el poll, lo asigna en elemento. El resultado de la expresión asignación es el valor asignado que luego es comparado con null.
Mi idea es que lo sólo lo sepan leer.

Sumario

- Excepciones:
  - Objeto de Java que representa un error.
  - Hay dos tipos: checked y unchecked.
  - Try-Catch para capturar excepciones.
- TDA Pila:
  - Secuencia con actualización LIFO
  - Java brinda clase java.util.Stack.
- TDA Cola:
  - Secuencia con actualización FIFO
  - Java brinda interfaz java.util.Queue y clase LinkedList.